

INTERVIEW PREPARATION GUIDE

TripLane

Collaborative Group Trip Planner

Architecture Diagrams · Full Route Reference
Database Model · Expected Interview Q&A

Full-Stack MERN Project
React 18 · Express · MongoDB · Node.js · Tailwind CSS

Contents

01	The 60-second pitch	Part I – The build
02	System architecture	Part I – The build
03	Folder map	Part I – The build
04	Every route, with its file path	Part I – The build
05	Request lifecycle	Part I – The build
06	Data model and indexes	Part I – The build
07	Voting mechanics	Part I – The build
08	Finalising, and what happens on a tie	Part I – The build
09	General & architecture	Part II – Q&A
10	Frontend	Part II – Q&A
11	Backend & API design	Part II – Q&A
12	Database	Part II – Q&A
13	Security	Part II – Q&A
14	The hard problems — your best material	Part II – Q&A
15	Performance & scale	Part II – Q&A
16	Testing & debugging	Part II – Q&A
17	Curveballs	Part II – Q&A
18	Numbers, names and one-liners	Part III – Reference

Every answer in Part II ends with the file path where that behaviour lives in the repository.

How the thing is built

Read this part once end to end. If you can redraw the four diagrams on a whiteboard from memory, you can survive almost any architecture question about this project.

01

The 60-second pitch

Learn this almost word for word. It is the answer to “tell me about your project”, and it seeds the follow-up questions you *want* to be asked.

SAY THIS

“TripLane is a collaborative trip planner for groups. The problem is that planning a trip in a group chat never converges — people suggest places, nobody tracks costs, and the decision drifts. In TripLane one person creates a trip with a budget range, travel window and currency, and shares a six-character invite code. Members join and suggest real destinations, searched through the Google Places API, each with a cost estimate, dates and a day-by-day activity list. Every member gets exactly one vote, which they can move but never duplicate. When the organiser finalises, the winner locks in and a live countdown to departure starts. Everything that happens is written to a group activity feed.”

Then offer the hook you are strongest on: *“The part I found most interesting was making duplicate votes actually impossible rather than just hidden in the UI, and deciding what the system should do when a vote ties.”* That pulls the interviewer toward section 14, which is your best material.

System architecture

Three tiers plus one external dependency. The key thing to point at: the browser never talks to Google — the server does, so the API key never reaches the client.

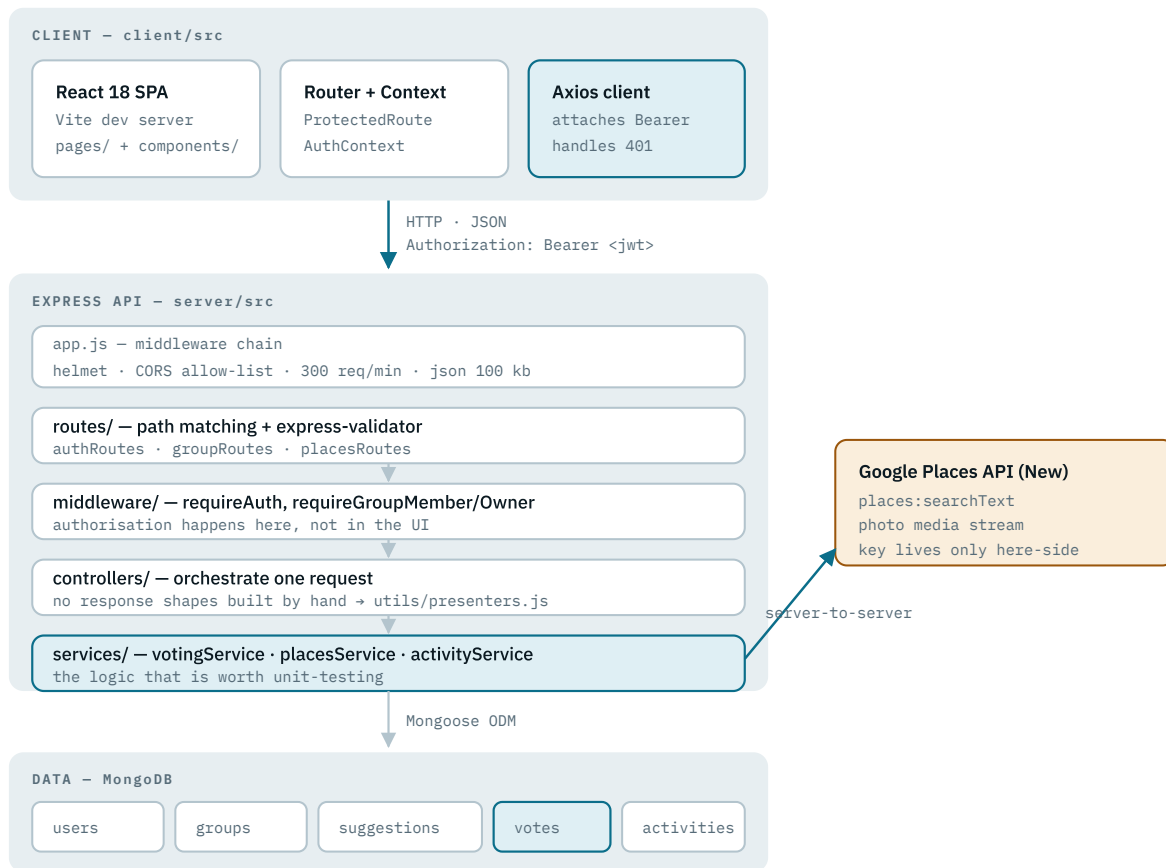


Figure 1 — Three tiers, one external dependency. Highlighted boxes are the pieces interviewers dig into: the Axios interceptor, the service layer, and the votes collection.

Folder map

Point at this when asked “how did you organise the code?”

```

Triplane/
├─ package.json          # root scripts: install:all, dev:server, dev:client
├─ README.md
├─ docs/interview-guide.html  # this document
├─
├─ server/
│   ├─ .env.example      # every variable, documented
│   ├─ .env              # real secrets (gitignored)
│   └─ src/
│       ├─ server.js      # boot: connect DB → listen → graceful shutdown
│       ├─ app.js         # express app + middleware chain
│       └─ config/
│           ├─ env.js      # loads .env, exits if MONGO_URI/JWT_SECRET missing
│           └─ db.js       # mongoose.connect
│       ├─ models/
│           ├─ User.js     # bcrypt pre-save hook, password select:false
│           ├─ Group.js    # members[] embedded, inviteCode unique
│           ├─ Suggestion.js # activities[] embedded, unique(group, placeId)
│           ├─ Vote.js     # unique(group, user) ← duplicate-vote guard
│           └─ Activity.js # append-only feed
│       ├─ middleware/
│           ├─ auth.js     # requireAuth: verify JWT, load req.user
│           ├─ groupAccess.js # requireGroupMember / requireGroupOwner
│           ├─ validate.js # turns express-validator result into 400
│           └─ errorHandler.js # notFoundHandler + single error formatter
│       ├─ controllers/
│           ├─ authController.js # register, login, me
│           ├─ groupController.js # create/join/update/finalize/reopen/members/feed
│           ├─ suggestionController.js
│           ├─ voteController.js # castVote, retractVote
│           └─ placesController.js # search, details, photo proxy
│       ├─ services/
│           ├─ votingService.js # tallyGroupVotes, getUserVote, getVoterIds
│           ├─ placesService.js # Google calls + offline fallback catalogue
│           └─ activityService.js # logActivity (never throws)
│       ├─ routes/
│           ├─ index.js      # mounts /auth /groups /places + /health
│           ├─ authRoutes.js
│           ├─ groupRoutes.js # the big one – groups, suggestions, votes
│           └─ placesRoutes.js
│       └─ utils/
│           ├─ ApiError.js   # status + machine-readable code
│           ├─ asyncHandler.js # forwards async rejections to errorHandler
│           ├─ presenters.js # the JSON contract, in one file
│           ├─ token.js      # sign / verify JWT
│           ├─ inviteCode.js # crypto-random, unambiguous alphabet
│           └─ currencies.js # 39 supported codes (server = authority)

```

```

|
└─ client/
    └─ vite.config.js          # proxies /api → localhost:5000
    └─ tailwind.config.js     # brand + ink palettes
    └─ src/
        └─ main.jsx           # BrowserRouter → ToastProvider → AuthProvider
        └─ App.jsx            # all routes + guards
        └─ api/
            └─ client.js       # Axios instance, interceptors, parseApiError
            └─ endpoints.js    # one wrapper per REST endpoint
        └─ context/
            └─ AuthContext.jsx # user, login, register, logout, initialising
            └─ ToastContext.jsx
        └─ hooks/useCountdown.js # ticks once/sec, stops when target passes
        └─ components/
            └─ ui/             # Button, Input, Modal, Badge, Avatar, ...
            └─ layout/         # Navbar, AppLayout, ProtectedRoute, PublicOnlyRoute
            └─ group/          # VotingPanel, SuggestionCard, Countdown, ...
        └─ pages/             # Landing, Login, Register, Dashboard, GroupWorkspace
        └─ utils/
            └─ format.js       # formatMoney, formatDateRange, relativeTime
            └─ currencies.js   # picker list + guessCurrency()

```

THE ONE-LINE ANSWER

“Routes validate, middleware authorises, controllers orchestrate, services hold the logic, presenters shape the JSON. Controllers never build a response object by hand, so every endpoint returns identical field names.”

Every route, with its file path

All API routes are mounted under `/api` by `server/src/app.js` → `server/src/routes/index.js`. The **Guards** column is the middleware chain in execution order — that ordering is itself an interview answer.

Auth — `server/src/routes/authRoutes.js`

METHOD	PATH	GUARDS → HANDLER	NOTES
POST	/api/auth/register	authLimiter → rules → validate → authController.register	Returns {token, user}
POST	/api/auth/login	authLimiter → rules → validate → authController.login	401 on bad credentials
GET	/api/auth/me	requireAuth → authController.me	Restores a session on refresh

Groups, suggestions and votes — `server/src/routes/groupRoutes.js`

This file opens with `router.use(requireAuth)`, so every route below is authenticated before anything else runs.

METHOD	PATH	GUARDS → HANDLER	WHO
GET	/api/groups	groupController.listMyGroups	any member
POST	/api/groups	groupBodyRules() → validate → createGroup	any user
POST	/api/groups/join	validate → joinGroup	any user
GET	/api/groups/:groupId	requireGroupMember → getGroup	member
PATCH	/api/groups/:groupId	requireGroupMember → requireGroupOwner → groupBodyRules({partial:true}) → validate → updateGroup	organiser
DELETE	/api/groups/:groupId	requireGroupMember → requireGroupOwner → deleteGroup	organiser
POST	/api/groups/:groupId/leave	requireGroupMember → leaveGroup	member
DELETE	/api/groups/:groupId/members/:userId	requireGroupMember → requireGroupOwner → removeMember	organiser
GET	/api/groups/:groupId/activity	requireGroupMember → validate → getActivityFeed	member
GET	/api/groups/:groupId/results	requireGroupMember → getResults	member
POST	/api/groups/:groupId/finalize	requireGroupMember → requireGroupOwner → validate → finalizeTrip	organiser
POST	/api/groups/:groupId/reopen	requireGroupMember → requireGroupOwner → reopenTrip	organiser
GET	/api/groups/:groupId/suggestions	requireGroupMember → listSuggestions	member
POST	/api/groups/:groupId/suggestions	requireGroupMember → suggestionBodyRules() → validate → createSuggestion	member
PATCH	/api/groups/:groupId/suggestions/:suggestionId	requireGroupMember → validate → updateSuggestion	author or organiser
DELETE	/api/groups/:groupId/suggestions/:suggestionId	requireGroupMember → validate → deleteSuggestion	author or organiser
POST	/api/groups/:groupId/suggestions/:suggestionId/activities	requireGroupMember → validate → addPlannedActivity	member
DELETE	/api/groups/:groupId/suggestions/:suggestionId/activities/:activityId	requireGroupMember → validate → removePlannedActivity	author or organiser

METHOD	PATH	GUARDS → HANDLER	WHO
POST	/api/groups/:groupId/suggestions/:suggestionId/vote	requireGroupMember → validate → voteController.castVote	member
DELETE	/api/groups/:groupId/suggestions/:suggestionId/vote	requireGroupMember → validate → voteController.retractVote	member

Places — server/src/routes/placesRoutes.js

METHOD	PATH	GUARDS → HANDLER	WHO
GET	/api/places/photo	placesLimiter → placesController.photo	public
GET	/api/places/search	requireAuth → placesLimiter → search	signed in
GET	/api/places/:placeId	requireAuth → placesLimiter → details	signed in

EXPECT THIS QUESTION

“Why is `/api/places/photo` public when everything else needs a token?” Because the browser loads it with `<img src=“...”>`, and an image tag cannot send an `Authorization` header. It is registered *before* `router.use(requireAuth)` in that file, deliberately. It stays safe because the photo reference is validated against a strict regex, the Google key never leaves the server, and the endpoint is rate limited to 60 requests a minute.

Health — server/src/routes/index.js

METHOD	PATH	PURPOSE
GET	/api/health	Liveness probe — { status, uptime }, no auth

Frontend routes — client/src/App.jsx

PATH	COMPONENT FILE	GUARD
/	client/src/pages/Landing.jsx	public
/login	client/src/pages/Login.jsx	PublicOnlyRoute
/register	client/src/pages/Register.jsx	PublicOnlyRoute
/dashboard	client/src/pages/Dashboard.jsx	ProtectedRoute
/groups/:groupId	client/src/pages/GroupWorkspace.jsx	ProtectedRoute
/groups	redirect → /dashboard	—
*	client/src/pages/NotFound.jsx	public

Request lifecycle

“Walk me through what happens when a request comes in” is asked in almost every backend interview. Trace one concrete request: `POST`

`/api/groups/:groupId/suggestions/:suggestionId/vote` .

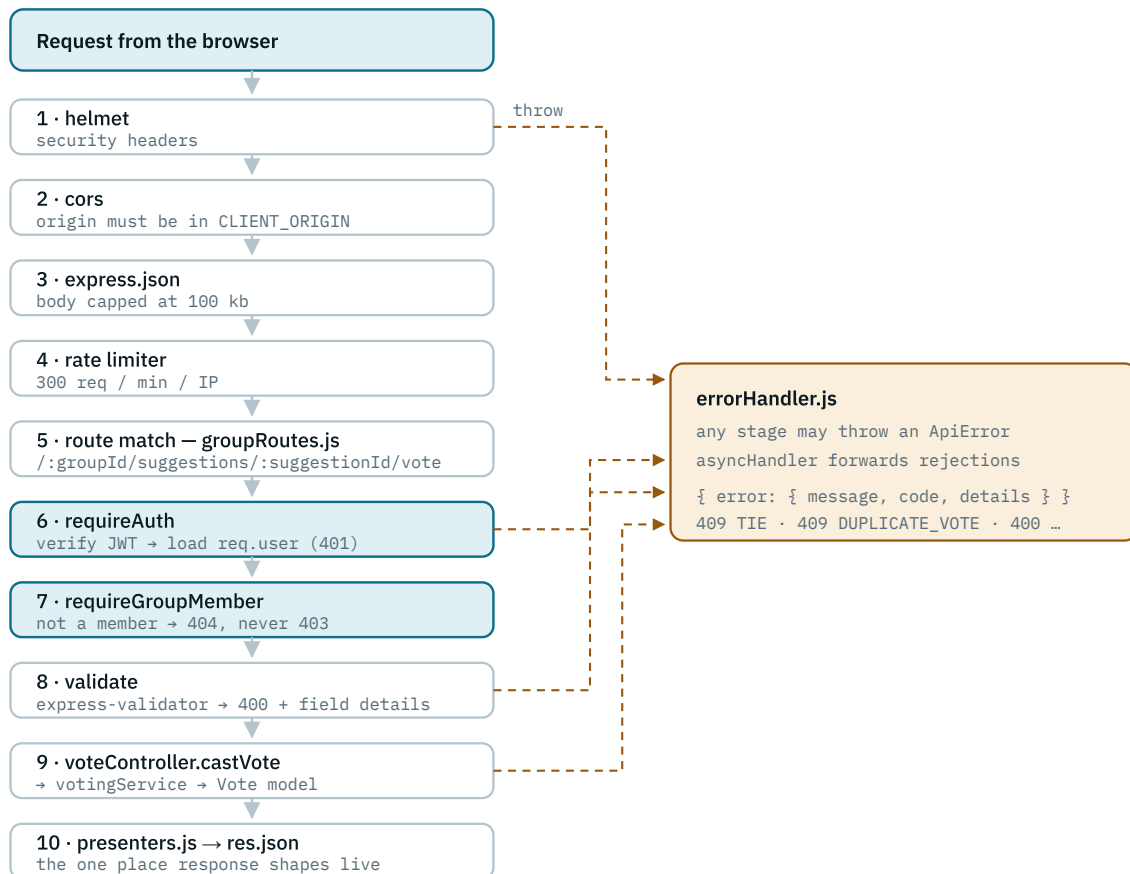


Figure 2 — Ten stages, one exit for errors. The dashed lines are what `asyncHandler` buys you: no try/catch in any controller.

SAY THIS

“Order matters. Cheap rejections happen first — headers, origin, body size, rate limit — so an abusive request never reaches the database. Authentication comes before authorisation, authorisation before validation, and validation before any business logic, so by the time my controller runs I know the caller is real, allowed, and the input is well-formed.”

Data model and indexes

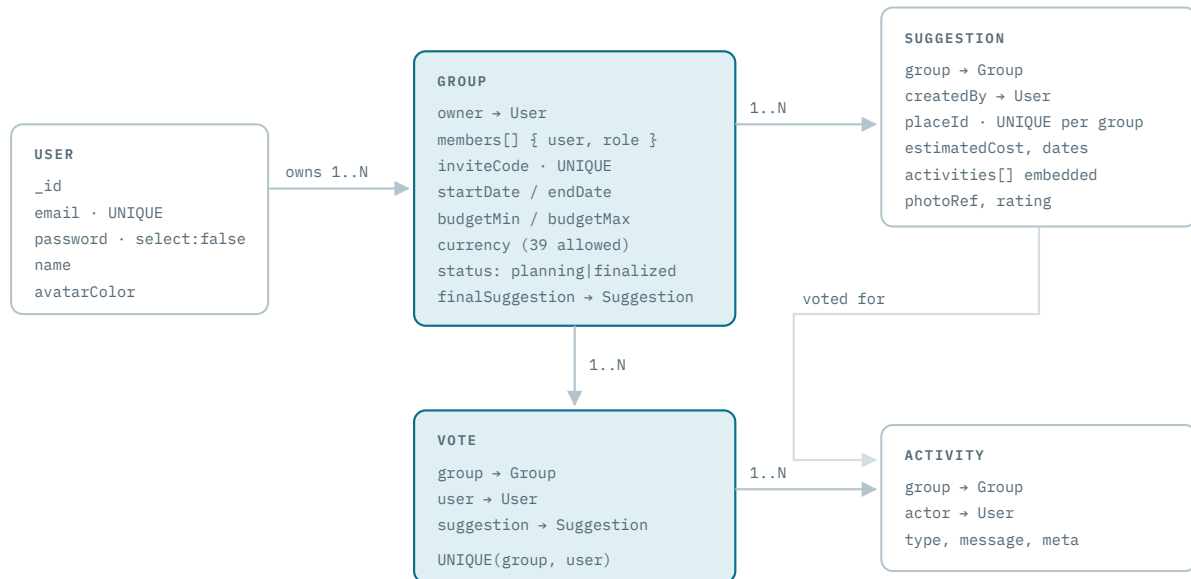


Figure 3 — Five collections. The two highlighted boxes carry the rules that make the product correct: one currency per group, and one vote per user per group enforced by a unique index.

INDEX	COLLECTION	WHY IT EXISTS
{ email: 1 } unique	users	One account per address; makes login lookup $O(\log n)$
{ inviteCode: 1 } unique	groups	Join-by-code lookup, and guarantees codes never collide
{ "members.user": 1 }	groups	Powers GET /api/groups — “trips I belong to”
{ group: 1 }	suggestions	Every workspace load lists suggestions by group
{ group: 1, placeId: 1 } unique, partial	suggestions	Same destination cannot be proposed twice and split the vote. Partial so manual entries (no placeId) are exempt
{ group: 1, user: 1 } unique	votes	The duplicate-vote guarantee. Even two simultaneous requests cannot create a second vote
{ group: 1, createdAt: -1 }	activities	Feed reads newest-first for one group

Voting mechanics

The rule: **one vote per member per group** — not per suggestion. Voting for a different destination *moves* your existing vote. This is the design decision the project is built around, so be ready to defend it.

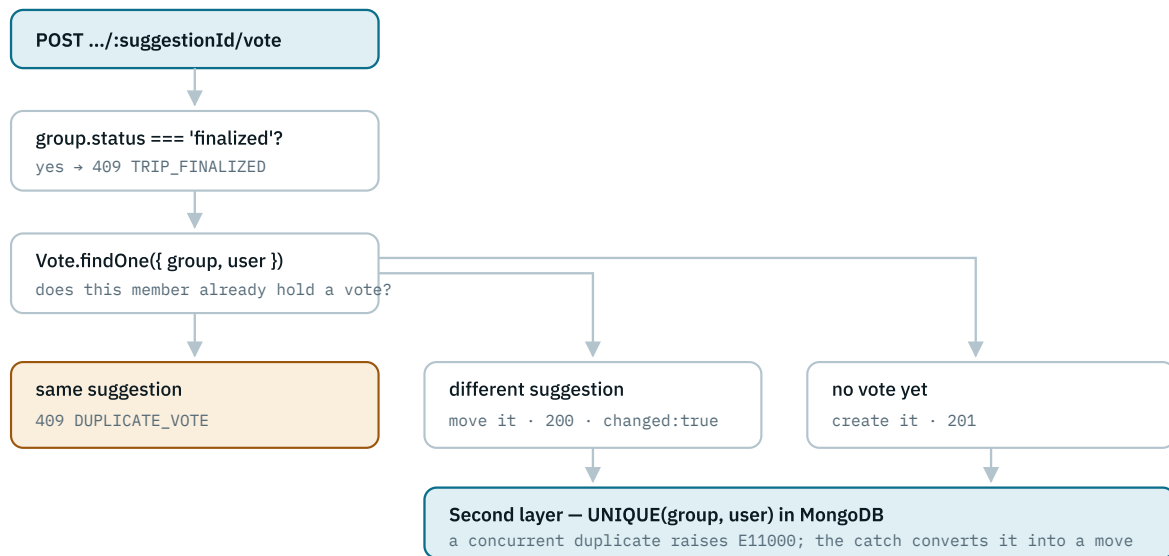


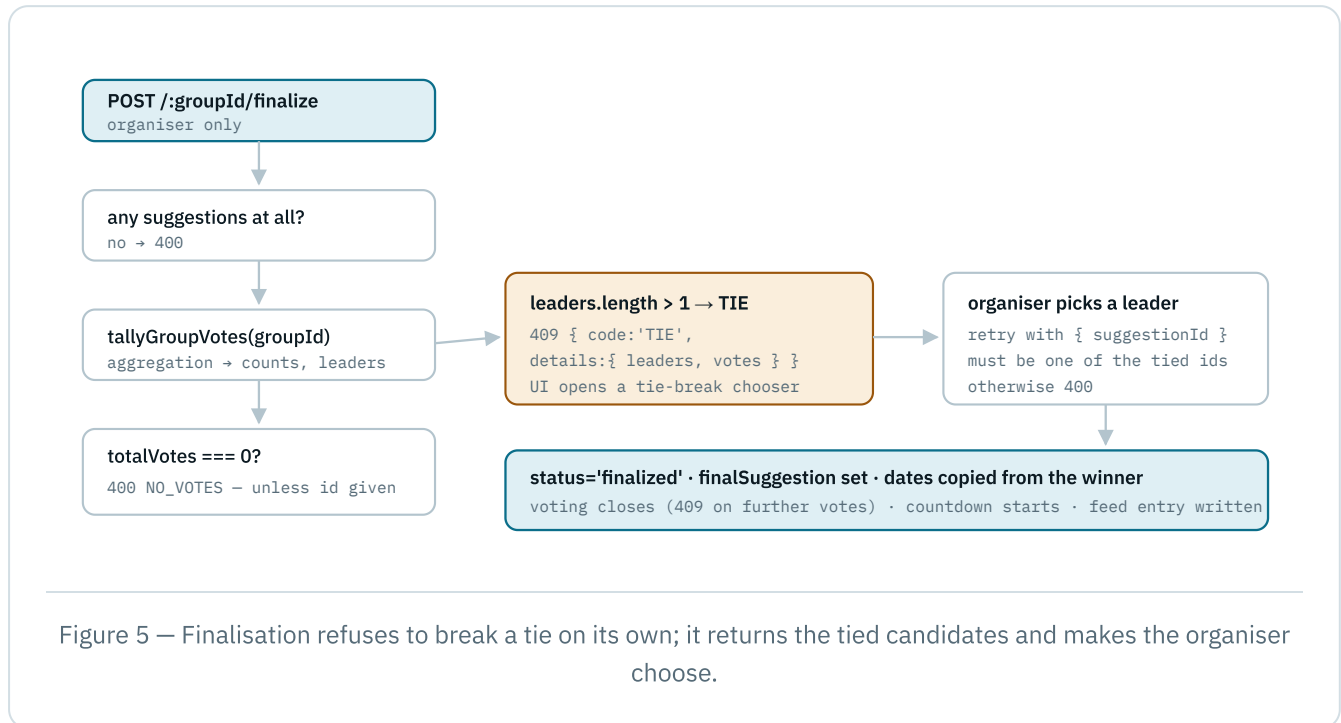
Figure 4 — Two layers of duplicate prevention: a friendly 409 for the normal case, and a database constraint that holds even when two requests race.

DEFEND THE DESIGN

“One vote per group keeps the tally meaningful — the percentages always sum to 100%, so ‘Kyoto is at 67%’ actually means two of three members. If I had allowed one vote per suggestion, a member could approve everything and the numbers would stop being a ranking.”

Finalising, and what happens on a tie

A tie is not an error — it is a state the product has to have an opinion about. Mine: the system refuses to guess, and hands the decision back to the organiser.



Questions you will actually be asked

Click any question to open the answer. Each one ends with the file to open if they push further — the fastest way to sound like you wrote the code is to know where it lives.

09

General & architecture

Q Why MERN? Why not something else?

Answer honestly, then justify per piece rather than saying “it’s popular”:

- **React** — the UI is a lot of small stateful pieces (vote buttons, countdown, modals) that re-render off shared state. A component model is the right fit.
- **Express** — I wanted explicit control of the middleware order for auth and authorisation; Express makes that chain visible in one file.
- **MongoDB** — a suggestion embeds a variable-length itinerary and a group embeds its member list. Those are documents, not rows. I still used references where I needed integrity, so it is not “schemaless” by accident.
- **Node** — one language across the stack; the workload is I/O-bound (DB + Google Places), which is exactly what the event loop is good at.

If pushed on Postgres: “Votes and members are genuinely relational and I’d get transactions for free. If this grew, that’s the migration I’d consider first.” Admitting that reads as senior, not weak.

WHERE `server/src/app.js` `server/src/models/`

Q Walk me through your architecture.

Use Figure 1, and narrate one request rather than listing folders: “The React SPA holds the session in Context; an Axios interceptor attaches the JWT. The Express app runs a fixed middleware chain — helmet, CORS, JSON limit, rate limit — then routes match and validate, middleware authorises, a controller orchestrates, a service holds the logic, and a presenter shapes the JSON. Mongoose talks to five collections. The only external call is Google Places, made server-side.”

WHERE `server/src/app.js` `server/src/routes/index.js`

Q Why the extra layers? Isn't a controller enough for a project this size?

Point at a concrete payoff: the vote tally is needed by three different endpoints — the workspace load, the results endpoint and finalisation. It lives in `votingService.tallyGroupVotes()` once. If it lived in a controller, I'd have copied an aggregation three times and they would have drifted.

Same for `presenters.js`: the client relies on `suggestion.totalCost` existing on every response. One file guarantees that.

WHERE `server/src/services/votingService.js` `server/src/utils/presenters.js`

Q What was the hardest part?

Don't say "CSS". Say: "Deciding what the system should do when the vote ties. Everything else had an obvious right answer; a tie has several defensible ones — random pick, earliest suggestion, organiser decides, re-vote. I chose to make the API refuse to guess: it returns 409 with the tied candidates, and the UI turns that into a chooser. The rule became 'the software never invents a decision the group didn't make'."

WHERE `server/src/controllers/groupController.js` → `finalizeTrip`

Frontend

Q Why Context API instead of Redux?

“There is exactly one piece of truly global state — the signed-in user. Everything else is page-local and fetched per screen. Redux would add a store, actions and reducers to hold one object. Context is the right size. If I added cross-page caching or optimistic updates I’d reach for React Query before Redux, because the real problem would be server-state caching, not global state.”

WHERE `client/src/context/AuthContext.jsx`

Q How do protected routes work — and why doesn’t refreshing a deep link log me out?

This is the detail that separates a copied tutorial from real work. `AuthContext` has an `initialising` flag that starts `true`. On mount it calls `GET /api/auth/me` with the stored token.

- While `initialising` is true, `ProtectedRoute` renders a spinner — it does **not** redirect.
- Only once the check resolves does it either render the page or send you to `/login`.

Without that flag, refreshing `/groups/abc123` would bounce to login for a moment, because on the very first render the user is still `null`. It also stashes the attempted path in router state so login returns you there.

WHERE `client/src/components/layout/ProtectedRoute.jsx` `client/src/context/AuthContext.jsx`

Q How is the token attached, and what happens when it expires?

A request interceptor in `client/src/api/client.js` reads the token from `localStorage` and sets `Authorization: Bearer ...`. A response interceptor watches for 401s and calls a handler that `AuthContext` registered, which clears the session once, centrally — instead of every component handling expiry.

One subtlety worth mentioning: 401s from `/auth/*` routes are skipped, because a failed login is a normal outcome and shouldn’t trigger a global “session expired” wipe.

WHERE `client/src/api/client.js`

Q Your destination search fires on every keystroke — how do you avoid hammering the API and getting out-of-order results?

Two separate problems, two fixes:

- **Volume** — a 350 ms debounce; the timer is cleared in the effect's cleanup on each keystroke.
- **Ordering** — every in-flight request carries an `AbortController` signal, and a newer keystroke aborts the previous one. Without this, a slow response for “go” could land after “goa” and overwrite the correct results.

Aborted requests surface as `ERR_CANCELED`, which is checked and ignored rather than shown as an error.

WHERE `client/src/components/group/PlaceSearch.jsx`

Q How does the countdown work? Does it leak?

`useCountdown` takes a target date and sets a one-second interval, returning days/hours/minutes/seconds. Two details to volunteer:

- The effect returns `clearInterval`, so navigating away kills the timer.
- If the target is already in the past, no interval is created at all — a finished trip doesn't re-render once a second forever.

WHERE `client/src/hooks/useCountdown.js`

Q After someone votes, how does the UI update? Why not optimistic updates?

The workspace calls a single `refresh()` that re-reads the group and the feed, so every panel — cards, standings, members, feed — is rendered from one server response and cannot disagree with itself.

Why not optimistic: a vote can legitimately be rejected by the server (trip just got finalised, a duplicate, the suggestion was deleted). Optimistically moving the bar and rolling it back would show a number that was never true. For a per-click action this cheap, correctness beat the few hundred milliseconds.

WHERE `client/src/pages/GroupWorkspace.jsx`

Q How did you keep components reusable?

A `ui/` folder of primitives with no business knowledge — `Button` , `Input` , `Modal` , `Badge` , `Avatar` , `ConfirmDialog` , `EmptyState` , `Spinner` . The one worth demoing is `Input` : it takes an `as` prop, so the same component renders an input, a textarea or a select while keeping identical label, hint, error and `aria-describedby` wiring. The currency picker is literally `<Input as="select">` .

WHERE `client/src/components/ui/Input.jsx`

Backend & API design

Q Why are suggestions and votes nested under groups in the URL?

Because they have no meaning outside a group, and nesting makes authorisation structural rather than optional: `requireGroupMember` runs on `:groupId` for every nested route, so there is no path to a suggestion that skips the membership check. A flat `/api/suggestions/:id` would need the same check re-implemented in every handler, and one forgotten line would be a data leak.

WHERE `server/src/routes/groupRoutes.js` `server/src/middleware/groupAccess.js`

Q You have no try/catch in your controllers. Why not?

Express 4 does not catch rejected promises from an async handler — it hangs the request. Every handler is wrapped in `asyncHandler`, a three-line utility that catches the rejection and calls `next(err)`, so all errors land in one place. That is also why controllers can just `throw` `ApiError.forbidden(...)` and trust the outcome.

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
```

WHERE `server/src/utils/asyncHandler.js` `server/src/middleware/errorHandler.js`

Q How does your client know *why* a request failed?

Every failure returns the same envelope, with a machine-readable code so the UI branches on meaning rather than parsing English:

```
{ "error": {  
  "message": "The vote is tied. Choose one of the tied destinations to break it.",  
  "code": "TIE",  
  "details": { "leaders": ["...", "..."], "votes": 2 } } }
```

The codes in use: `VALIDATION_ERROR`, `ALREADY_MEMBER`, `DUPLICATE_VOTE`, `TRIP_FINALIZED`, `NO_VOTES`, `TIE`, `DUPLICATE`. Concrete payoff: on `ALREADY_MEMBER` the join dialog navigates you into the group instead of showing a dead-end error.

WHERE `server/src/utils/ApiError.js` `client/src/api/client.js` → `parseApiError`

Q Where do you validate input?

Three layers, deliberately: **client** mirrors the rules for instant feedback (never trusted); **express-validator** in the route file rejects malformed input before any logic runs; **Mongoose schema** constraints are the last line at the database boundary. The error handler maps a Mongoose `ValidationError` into the same 400 shape as express-validator, so the client sees one format either way.

WHERE `server/src/routes/groupRoutes.js` `server/src/middleware/validate.js`

Q Talk me through your rate limiting.

- **300/min globally** — a blanket safety net (`app.js`).
- **60/min on `/api/places`** — Google bills per call, so this is a cost control as much as a security one.
- **20 per 15 min on auth in production** (looser in development, so seeding and manual testing don't trip it) — this is the anti-credential-stuffing one.

Note `app.set('trust proxy', 1)` — behind a reverse proxy the limiter must read the forwarded IP or it throttles everyone as one client.

WHERE `server/src/app.js` `server/src/routes/authRoutes.js`

Database

Q Why are itinerary activities embedded but votes a separate collection?

This is the classic Mongo modelling question — answer with the access pattern:

- **Embedded** (planned activities inside a suggestion, members inside a group): always read together with the parent, bounded in size, never queried on their own. Embedding makes the workspace load one query instead of three.
- **Referenced** (votes): I need to aggregate across them, count them per suggestion, and — crucially — enforce a *unique constraint* on (group, user). You cannot put a unique index on a field inside an array element in the way I need. The constraint drove the collection.

WHERE `server/src/models/Suggestion.js` `server/src/models/Vote.js`

Q How do you count the votes?

An aggregation, not a JavaScript loop — the counting happens in the database:

```
Vote.aggregate([
  { $match: { group: groupId } },
  { $group: { _id: '$suggestion', count: { $sum: 1 } } },
])
```

From that one pass the service derives the per-suggestion counts, the total, the top count, the list of leaders and the `isTie` flag. Everything about the vote state comes from a single round trip.

WHERE `server/src/services/votingService.js` → `tallyGroupVotes`

Q What happens to suggestions and votes when a group is deleted?

MongoDB has no foreign keys, so the cascade is explicit in `deleteGroup`: suggestions, votes and activity entries for that group are removed, then the group itself. The same care applies elsewhere — removing a member or deleting a suggestion also deletes the affected votes, otherwise the tally would count ballots from people who left.

Be honest about the gap: those deletes are not in a transaction. If the process died mid-cascade you could orphan documents. With a replica set I'd wrap it in a Mongo session; on a standalone dev server transactions aren't available. I verified the cascade leaves nothing behind by querying the collections directly after the test run.

WHERE `server/src/controllers/groupController.js` → `deleteGroup`

Q How do the invite codes work? Could two groups get the same one?

Six characters from a 32-character alphabet that deliberately excludes `0/O` and `1/I`, so a code can be read aloud without confusion. That is $32^6 \approx 1.07$ billion combinations, drawn from `crypto.randomBytes` rather than `Math.random`.

Collisions are impossible to persist because `inviteCode` carries a unique index — and creation retries up to five times on an `E11000`, so the rare collision is invisible to the user rather than a 500.

WHERE `server/src/utils/inviteCode.js` `server/src/controllers/groupController.js`

Security

Q How are passwords stored?

bcrypt with a cost factor of 12, hashed in a Mongoose `pre('save')` hook so no controller can ever write a plaintext password by accident. The field is declared `select: false`, so it is excluded from query results by default — login has to ask for it explicitly with `.select('+password')`.

If asked why bcrypt and not SHA-256: “SHA is fast, which is exactly wrong for passwords. bcrypt is deliberately slow and salted per user, so identical passwords produce different hashes and brute-forcing is expensive. Cost 12 is roughly a quarter-second per hash.”

WHERE `server/src/models/User.js`

Q Why does a wrong email and a wrong password return the same message?

Because different messages turn the login form into an account-enumeration oracle: an attacker could discover which email addresses are registered. Both cases return 401 “Incorrect email or password”. There is a test asserting exactly that string, so a well-meaning future change can’t quietly reintroduce the leak.

WHERE `server/src/controllers/authController.js` → `login`

Q Where do you store the JWT, and what’s the trade-off?

In `localStorage`, attached by the Axios interceptor. State the trade-off yourself before they do: **localStorage is readable by any script on the page, so it is vulnerable to XSS**; an `httpOnly` cookie would be immune to that but needs CSRF protection and complicates cross-origin dev. For this project I chose the simpler model and mitigated the input side (validation everywhere, React escaping output by default, helmet headers).

Also worth volunteering: `requireAuth` re-loads the user from the database on every request rather than trusting the token payload — so a deleted account’s token stops working immediately instead of staying valid until expiry.

WHERE `server/src/middleware/auth.js` `client/src/api/client.js`

Q Someone who isn't in a group requests it. What do they get, and why?

404, not 403. A 403 would confirm the group exists, which leaks information — someone could probe IDs and learn which trips are real. The middleware treats “not a member” and “does not exist” identically.

The authorisation itself is entirely server-side. Hiding a button in the UI is a courtesy; the API rejects the call regardless, and there are tests that call every organiser-only endpoint as a plain member and assert 403, and as an outsider and assert 404.

WHERE `server/src/middleware/groupAccess.js`

Q How do you protect the Google API key?

It never reaches the browser. The client calls `/api/places/search` on my own server; the server calls Google with the key in a header. Photos are the interesting case — Google serves them from a URL that needs the key, so rather than hand that URL to the browser, the server streams the image back through `/api/places/photo?ref=...`.

That proxy is the one unauthenticated route, because `<img>` cannot send a bearer token. It is protected by a strict regex on the photo reference plus the rate limiter. Say that out loud — an interviewer who spots the public route will be impressed you already know why.

WHERE `server/src/services/placesService.js` `server/src/routes/placesRoutes.js`

Q What else did you harden?

- `helmet` for security headers.
- CORS restricted to an explicit allow-list from `CLIENT_ORIGIN`, not `*`.
- JSON body capped at 100 kb.
- Secrets only in `.env`, which is gitignored; `.env.example` documents every variable with no real values.
- The server **exits at boot** if `MONGO_URI` or `JWT_SECRET` is missing — no silently defaulted signing secret.
- The Mongo duplicate-key number (11000) is mapped to a friendly `DUPLICATE` code so raw driver internals aren't echoed to clients.

WHERE `server/src/config/env.js` `server/src/app.js`

The hard problems — your best material

These are the stories that make you sound like an engineer rather than someone who followed a tutorial. Have all four ready.

Q How do you stop someone voting twice?

Two layers, and the second is the one that matters:

- **Application check** — the controller looks for an existing vote and returns `409 DUPLICATE_VOTE` if you're re-voting for the same destination. This exists for the error message, not for safety.
- **Database constraint** — a unique index on `{ group, user }`. Two requests firing at the same millisecond both pass the check, but only one insert can succeed; the other raises `E11000`, which is caught and converted into a vote move.

The line to say: *"A check-then-write is a race condition unless the database enforces the invariant. The index is the real guarantee; the check is just for a nicer message."*

WHERE `server/src/models/Vote.js` `server/src/controllers/voteController.js`

Q Tell me about a bug you found and fixed.

The strongest story in the project — a silent library-version bug:

- **Symptom:** while testing the currency feature, sending `currency: ''` was rejected, but `currency: null` was *accepted* and stored as null.
- **Investigation:** I had written `optional({ nullable: true, checkFalsy: true })`. I reproduced it in an isolated 20-line Express app and found `express-validator` v7 had replaced those flags with `optional({ values: 'falsy' })` — the old options are accepted and **silently ignored**. No warning, no error.
- **Blast radius:** the same stale option was on every optional date field — which meant *clearing the trip dates in settings was broken*, because the UI sends `null` and validation rejected it. A real user-facing bug I hadn't noticed.
- **Second cause:** the null was slipping past Mongoose too, because enum validators skip null, and a destructuring default only fires on `undefined`.
- **Fix:** corrected the option across all rules, and normalised falsy currency to the default in the controller so the invariant holds no matter what the client sends.
- **Prevention:** I also found that `.map(rule => rule.optional())` on the PATCH routes was *resetting* those options, so I rebuilt the rules through a factory function that takes a `partial` flag. Then I wrote regression tests for clearing dates with both `null` and `''`.

That answer demonstrates reproduction, root-cause analysis, blast-radius thinking and prevention — which is exactly the loop interviewers are testing for.

WHERE `server/src/routes/groupRoutes.js` → `groupBodyRules()`

Q Why doesn't your app convert currencies?

Because a shared budget only works if everyone reads the same number. The currency belongs to the trip, not the member: the organiser picks one of 39 supported codes and every amount in that trip is formatted with it. A per-member display currency would need a live exchange-rate feed, and rates that drift would mean two members seeing different “over budget” verdicts on the same suggestion.

Two details worth adding: the picker defaults to the currency implied by the browser's locale (an `en-IN` browser gets INR rather than dollars), and if the organiser changes it later the settings dialog explicitly warns that amounts are *relabelled, not converted* — because pretending otherwise would silently corrupt the numbers.

WHERE `server/src/utls/currencies.js` `client/src/utls/currencies.js`

Q What happens if the Google Places API is down or you have no key?

Failures are translated rather than leaked: a 401/403 from Google becomes “check the API key and that Places API (New) is enabled”, a 429 becomes a rate-limit message, anything else a generic 502. The raw Google error is logged server-side but never forwarded to the browser.

With no key configured at all, search falls back to a built-in catalogue of twelve destinations and marks the response `source: "fallback"`, which the UI surfaces as a small amber note. That was a deliberate choice so the project stays fully runnable for a reviewer without a billing-enabled Google account.

WHERE `server/src/services/placesService.js`

Performance & scale

Q Loading a workspace shows a group, members, suggestions and a tally. How many queries is that?

Not one per suggestion — that would be the N+1 trap. The workspace endpoint runs the independent reads concurrently with `Promise.all`: the suggestions (with authors populated in the same round trip), the vote aggregation, the caller's own vote, and the list of who has voted. Members come back on the group document itself because they are embedded.

WHERE `server/src/controllers/groupController.js` → `getGroup`

Q How would you scale this to 100,000 users?

Answer in order of what actually binds first:

- **Stateless already** — the JWT means no server-side sessions, so I can run N Express instances behind a load balancer today.
- **Move the rate-limit store to Redis** — the in-memory limiter is per-process, so with 4 instances the effective limit is 4×. This breaks first and is easy to miss.
- **Cache Places results in Redis** — searches repeat heavily (“Goa”, “Bali”) and Google bills per call. A short TTL cuts both latency and cost.
- **Replica set + read preference** for the read-heavy workspace endpoint, which also unlocks real transactions for the cascade deletes.
- **Websockets** for live vote updates instead of refetching.
- Sharding only if a single group ever got enormous — realistically groups are tens of people, so the data per group stays tiny. Say this: it shows you size the problem instead of reciting buzzwords.

Q Why isn't the app real-time?

Be direct: it isn't, and I know it. Your own actions refresh immediately; another member's vote appears on your next load. For a planning tool where decisions take days, polling or a refresh was acceptable, and websockets would have added infrastructure the MVP didn't need. The upgrade path is clean, though — every mutation already writes an activity entry, so a Socket.io room per group could broadcast those same events with no data-model change.

Testing & debugging

Q How did you test this?

End-to-end against a live server and a real MongoDB, not mocks — **118 assertions** in total (93 in the main suite, 25 more for currency and the validation regressions). They cover registration and login, token rejection, invite codes, member vs organiser vs outsider authorisation, suggestions, itinerary activities, voting, duplicate rejection, vote moving, tie detection, tie-breaking, finalisation, reopening, the activity feed, membership changes and cascade deletion.

I also verified database integrity directly — no orphaned documents after the cascade tests, every password bcrypt-hashed, and the unique indexes actually present — and drove the main user journeys in a real browser.

Own the gap: “These are integration tests. There are no unit tests, and no CI. If I continued, I’d put Jest + Supertest around `votingService` first, because the tally and tie logic is the part where a subtle regression would be hardest to spot by eye.”

Q A user says “I voted but the count didn’t change.” How do you debug it?

Narrate a path, don’t guess a cause — work along the request:

- **Network tab** — did the POST fire, and what status? A 409 with `DUPLICATE_VOTE` or `TRIP_FINALIZED` answers it immediately.
- **If 2xx** — check the response body’s `voting.counts`. If the server counted it, the bug is client-side: the refresh didn’t run or state didn’t re-render.
- **If the count is wrong server-side** — query the votes collection directly for that group and user. Is there a vote row? Does it point at the expected suggestion?
- **If the row is right but the tally is wrong** — the bug is in the aggregation’s `$match`, most likely an ObjectId-vs-string mismatch, which is the classic Mongo aggregation trap.

Ending on a named likely culprit is what makes this answer land.

Curveballs

Q What would you do differently if you started again?

- **Write tests alongside the code.** I tested thoroughly, but at the end. The express-validator bug had been sitting there silently the whole time and a test written on day one would have caught it immediately.
- **Consider Postgres.** Votes and memberships are relational, and I'd get transactions and real foreign keys instead of hand-written cascades.
- **Pick the token storage more deliberately** — httpOnly cookies with CSRF protection is the better default.

Q What's the weakest part of your code?

Never say “nothing”. Say: “The cascade deletes aren't transactional, so a crash mid-delete could orphan documents. It's the one place where a failure leaves the database inconsistent, and it's the first thing I'd fix with a replica set and a Mongo session.” A specific, correct weakness with a named fix reads as self-awareness.

Q Two organisers finalise at the same instant. What happens?

Reason it through out loud rather than bluffing: there is one organiser per group, so this needs the same person in two tabs. Both requests would read `status: 'planning'` and both could write. The second write wins and the feed would show two finalisation entries.

Then give the fix: a conditional update — `findOneAndUpdate({ _id, status: 'planning' }, { status: 'finalized' })` — so the status change is atomic and the loser gets the “already finalised” 409 that already exists for the sequential case. Recognising a real race you didn't handle, and knowing the one-line fix, scores better than claiming it can't happen.

Q How much of this did you write yourself?

Answer honestly — and note that the defence against this question is simply knowing your own codebase. Everything in this guide is checkable in the repo; the fastest way to prove ownership is to open `voteController.js` and walk them through the duplicate-vote race live, then show the unique index in `Vote.js` that makes it safe. Depth of explanation is the proof, not the claim.

One-page cheat sheet

Read this on the way in.

18

Numbers, names and one-liners

NUMBERS TO KNOW

- bcrypt cost **12**, JWT expiry **7d**
- Invite code: **6 chars**, 32-char alphabet, **32⁶ ≈ 1.07 B**
- **5** collections · **27** endpoints · **4** unique constraints
- Rate limits: **300**/min global, **60**/min places, **20**/15min auth
- **39** currencies · **118** test assertions

ERROR CODES

- **VALIDATION_ERROR** — 400 + per-field details
- **ALREADY_MEMBER** — 409, UI redirects in
- **DUPLICATE_VOTE** — 409, same destination twice
- **TRIP_FINALIZED** — 409, voting closed
- **TIE** — 409 + tied leaders
- **NO_VOTES** — 400, nobody voted yet

FILES THEY'LL ASK YOU TO OPEN

- `server/src/controllers/voteController.js`
- `server/src/models/Vote.js`
- `server/src/middleware/groupAccess.js`
- `server/src/services/votingService.js`
- `client/src/context/AuthContext.jsx`

SENTENCES THAT LAND

- “A check-then-write is a race unless the database enforces the invariant.”
- “Non-members get 404, not 403 — a 403 confirms the group exists.”
- “The software never invents a decision the group didn’t make.”
- “Hiding a button is a courtesy; the API rejects it regardless.”
- “The constraint drove the collection.”

LAYERING, IN ONE BREATH

Routes validate → middleware authorises → controllers orchestrate → services hold logic → presenters shape JSON.

IF YOU BLANK

Fall back to tracing one request end to end. It buys thinking time and shows you understand the whole stack, whatever the original question was.

LAST THING

Before the interview, run the app and click through once: register, create a trip, suggest a place, vote, force a tie, finalise, watch the countdown. Talking about software you used an hour ago sounds completely different from talking about software you last saw a month ago.